

Team #47

HintonMarket

Subsystem Design

Deliverable 2 Part 1

Raymond Liu 101264487
Ryan Zhang 101264124
Rodney Shen 101277506
Daniel Byshkin 101348289

April 5th, 2026

1. D1 Version

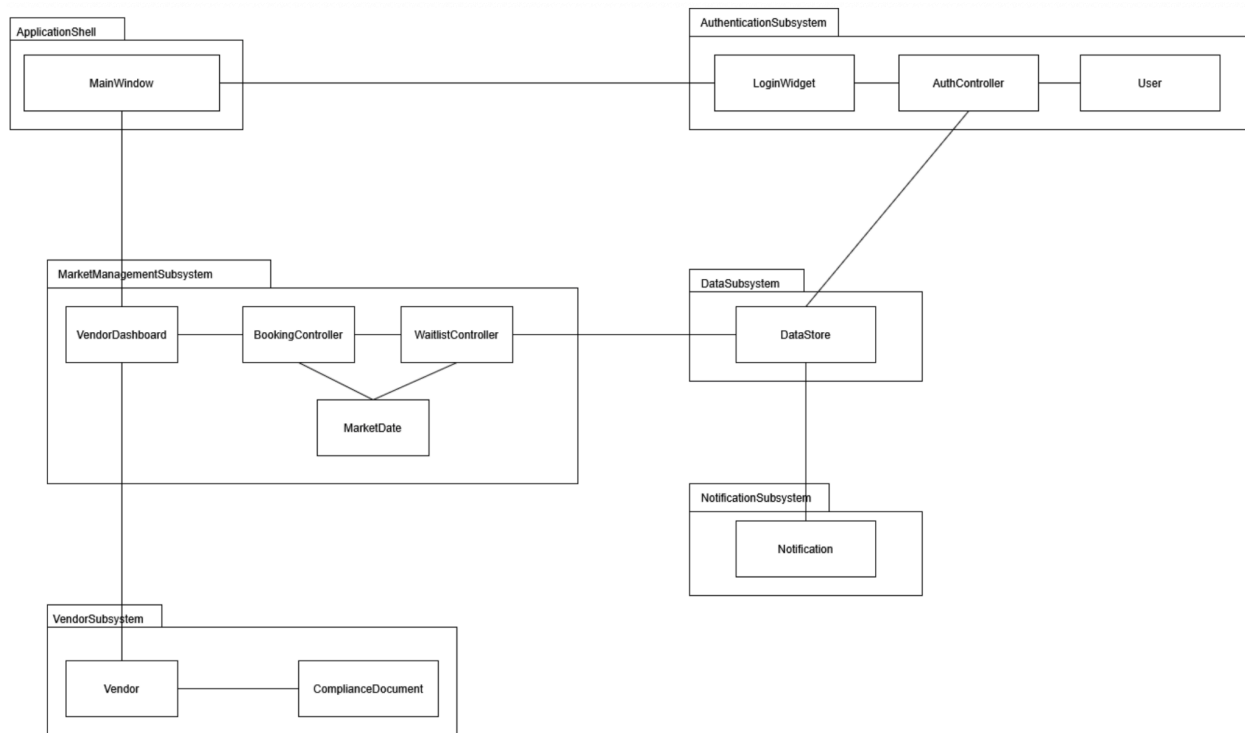


Figure 1: HintonMarket System Decomposition Showing Subsystem Organization and Dependencies D1

2. Updated Version

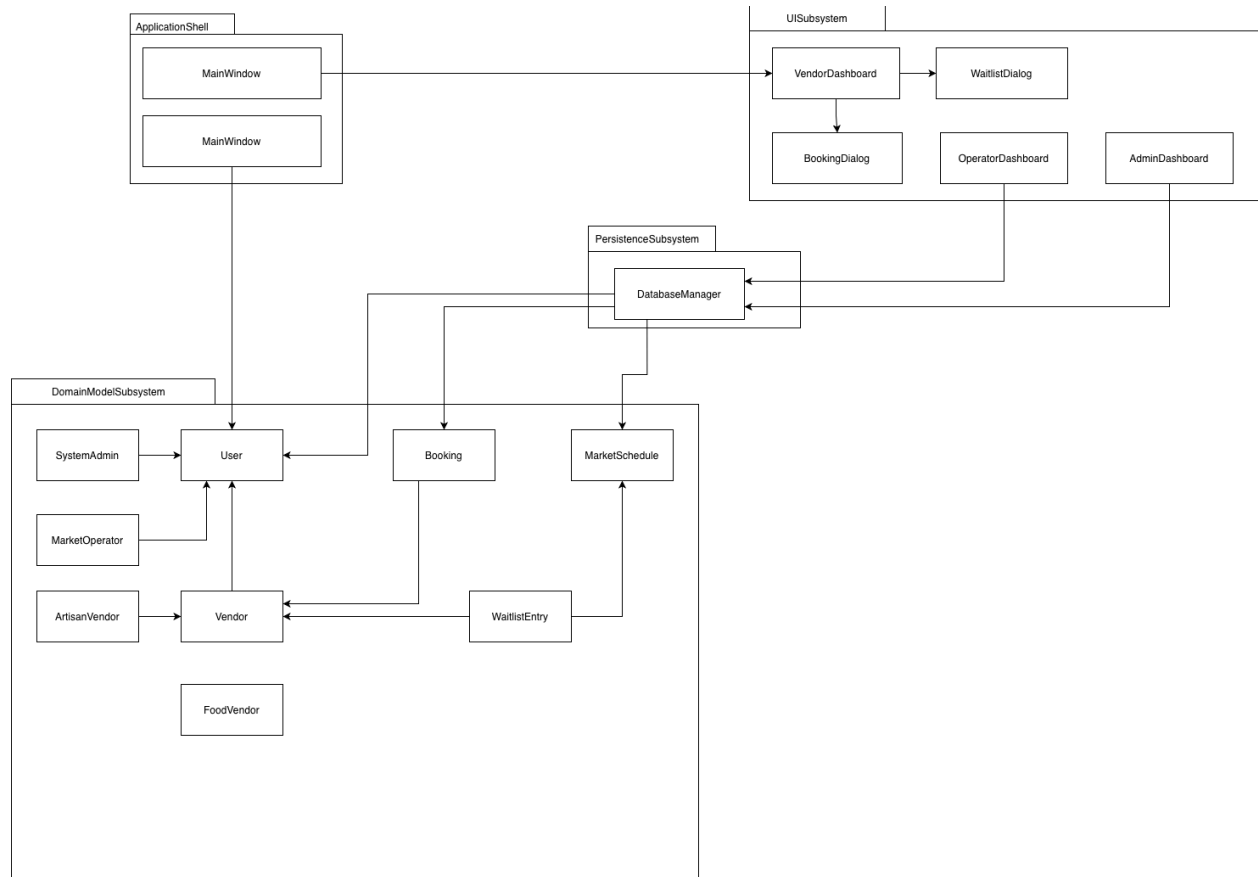


Figure 2: HintonMarket System Decomposition Showing Subsystem Organization and Dependencies D2

Written Explanation

1. What Changed from D1 to D2

New Subsystem: PersistenceSubsystem

The most significant structural addition is the PersistenceSubsystem, containing a single class: DatabaseManager. In D1, all data was stored in memory (C++ QList and QMap structures held within the UI widgets themselves). In D2, a SQLite database replaces in-memory storage entirely, and DatabaseManager, which is implemented as a Singleton, is the sole access point for all reads and writes. Every view class now routes data through it.

New Class: WaitlistEntry

In D1, waitlist state existed implicitly as entries stored inside MarketDate or similar containers. In D2, WaitlistEntry is a formal domain model class in DomainModelSubsystem, with its own

identity (id), FK references (vendorId, scheduleId), category, joinDate, and position. This makes FIFO ordering and per-vendor/per-schedule queries explicit and persistent.

New UI Classes: OperatorDashboard and AdminDashboard

D2 introduces two new widgets in UISubsystem:

- **OperatorDashboard**: allows the market operator to book stalls, cancel bookings, and remove vendors from waitlists on behalf of vendors (new D2 requirement).
- **AdminDashboard**: displays system-wide statistics (all users, total bookings, total waitlist entries).

These are created by MainWindow at startup and swapped in via a QStackedWidget depending on the logged-in user's role.

Subsystem Organization Now Matches the Code

In D1, the grading feedback noted that the diagram's subsystem organization did not match the code. In D2, the four subsystems map directly to the source folder structure:

Subsystem	Folder
ApplicationShell	src/ui/mainwindow.*, src/ui/loginwindow.*
UISubsystem	src/ui/ (dashboards, dialogs)
DomainModelSubsystem	src/models/
PersistenceSubsystem	src/database/

Associations Now Use Directed Arrows

D1 grading noted that associations were drawn without arrowheads (treating them as bidirectional). In D2, all associations are directed (filled arrowhead at the target), reflecting the actual direction of dependency in the code.

2. Impact on Cohesion and Coupling

Cohesion

- DomainModelSubsystem has high cohesion. All classes in src/models/ are pure data representations with no Qt Widget or database dependencies. Each class is responsible for one concept (Booking, WaitlistEntry, MarketSchedule, etc.).
- PersistenceSubsystem has high cohesion. DatabaseManager is solely responsible for all data persistence. It abstracts away SQL entirely and presents a clean, typed API returning domain objects. Nothing else in the system touches SQL.
- UISubsystem has moderate cohesion. Each dashboard is responsible for one role's complete workflow (e.g., VendorDashboard handles booking, waitlist, and notifications all in one class). This is reasonable for a desktop Qt application but means individual dashboards are larger classes with multiple responsibilities.

Coupling

- Reduced coupling between UI and data. In D1, views held data directly or called functions on other view classes. In D2, all views are decoupled from each other and from raw data structures. They all talk exclusively to DatabaseManager, which is a stable, well-defined interface.
- Increased coupling to DatabaseManager. All three dashboards (VendorDashboard, OperatorDashboard, AdminDashboard) and LoginWindow all directly depend on DatabaseManager::instance(). This is an acceptable trade-off given the Singleton pattern: if the persistence interface changes, only DatabaseManager needs updating, not the database schema itself.
- Domain model classes have zero coupling to UI or persistence. User, Vendor, Booking, etc. are unaware of DatabaseManager or any Qt Widget class. They are returned as value objects by DatabaseManager, consumed by the view layer, and discarded or passed as pointers via Qt signals.
- Inheritance Coupling within DomainModelSubsystem. The inheritance hierarchy (User → Vendor → FoodVendor / ArtisanVendor; User → MarketOperator / SystemAdmin) is tighter than in D1 (where these subclasses were flatter). This is appropriate coupling, and it reflects a genuine is-a taxonomic relationship.

3. Architecture Identification

The D2 system follows a Repository (2-tier) architecture, not MVC.

Why not MVC: There are no separate Controller classes. The `QWidget`-based view classes (VendorDashboard, OperatorDashboard, etc.) handle both display logic and user interaction (slot/signal handling) in a single class. Qt's signal/slot mechanism blurs the traditional View/Controller boundary in desktop applications.

Why Repository: DatabaseManager acts as a central Repository, and is a single object that mediates between the application and the database, abstracting the persistence mechanism. Views call the repository directly to load and save data. This is consistent with the Repository

pattern described in the course material and is appropriate for a standalone desktop application with a single concurrent user.

Tier breakdown:

1. **Presentation tier.** ApplicationShell + UISubsystem (Qt Widgets, handles input and display)
2. **Persistence tier.** PersistenceSubsystem (DatabaseManager, QSqlDatabase/SQLite)
3. **Domain model.** DomainModelSubsystem (plain data objects, returned from tier 2 and consumed by tier 1)